Another Eye

In fulfillment of Distinction in Computer Science

By Sage Labesky

November 2025

Advisor: Matt J. Rutherford

https://github.com/SageLabesky/Distinction_Project_Labesky

Abstract

Author: Sage Labesky

Title: Another Eye

Advisor: Matt J. Rutherford

Degree Date: November 2025

This project provides a unique implementation of an automated visual, textual, and audio-based journal. It uses both hardware and software solutions, combining a Raspberry Pi with AI to generate this journal. The Raspberry Pi hosts the functionality of the project. It interfaces with a camera and external button to allow photographs to be taken. It also hosts a webserver to display the photographs it takes. An external AI API is used to generate text and audio to describe the images taken on the device.

This device provides a way to take photographs to form this visual journal with minimal user input. The device can take photographs at regular intervals. This is a unique implementation that could be used by the user to create a summary of their day. This is useful for both social media and for general recordkeeping by the user.

Table of contents

Project Functionality Overview	1
Project Purpose	1
Hardware Solution	2
Software Solution	3
Implementation Challenges	5
Conclusion	6
Bibliography	7

Project Functionality Overview

Another Eye is a project that uses AI image recognition to describe photographs taken by a small device. It then outputs the descriptions to the user as audio. The device can take photographs when the user pushes a button or at a specified interval. The program currently runs from the command line where the user can enter the interval at which they would like photographs to be taken. Photographs will then be taken at the specified interval or when the user presses a button. When an image is taken, it is sent to an AI image analyzer and output to the user as audio. The image, text description, and audio description are also saved in the device and can be viewed on a built-in web server by any other devices on the local network. Figure 1 shows the general flow of the project functionality.

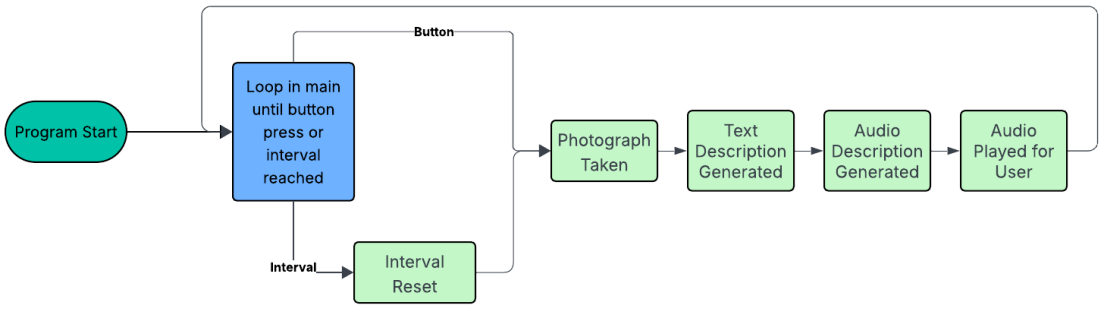

Figure 1: This is a flowchart of the main functionality of the project.

Project Purpose

This device was built primarily with entertainment in mind. It has two purposes. One purpose is to serve as a visual journal. The user can take quick photographs of interesting things they see throughout their day and look back on these images later. Setting the photographs to be taken at an interval serves a similar purpose to the social media app "Be Real." [1] The social media app is meant to have users share candid photographs of what they are doing at a random time of the day, but the user still has to consciously take the photograph themself. With my device, the photographs can be taken entirely independently of user input which can create an authentic visual journal with completely random snapshots of someone's day. The user can then look back on them as an unconsciously written diary.

The second purpose of this device is to aid people who have vision difficulties in understanding what specific things around them are. The program's running speed is not quick enough for the tool to fully guide a person who cannot see. It is more useful if they have a specific thing they are trying to identify. An example of this is a person who is

extremely nearsighted going on a hike with friends. If the friends see an interesting rock or view in the distance, the nearsighted individual can use the device to both get an accurate description of the view and also get a picture of the view that they can then look at.

In any case, the device is meant to be a tool for entertainment. Creating a visual journal and providing descriptions of the photographs allows it to provide this entertainment value.

Hardware Solution

The device providing the processing power is a Raspberry Pi Zero W2. [2] This is the smallest model of Raspberry Pi. This means that it has very limited processing power, only having 512 megabytes of RAM. The device is meant to be small so that it can be easily carried by a user throughout the day. The device must be connected to power via a micro-USB port. Two peripherals interface with the Raspberry Pi to give the program its functionality.

One of these peripherals is a camera. This device uses a 120 degree focal andgle spy camera from AdaFruit. [3] This camera is designed to connect directly to the Raspberry Pi Zero using a ribbon. It takes photographs at a very wide angle which is helpful for capturing complicated images.

The second peripheral is a button [5] that makes use of the Pi Zero's built in GPIO pins. These pins allow electrical signals to be sent and received to and from the Pi. The button uses a pull up resistor to send a high signal to the Pi when the button is pressed. A wire is connected to pin 1 of the Pi Zero which outputs 3.3 volts of power constantly. The wire leads to a resistor which leads to the button. The other side of the button is connected with a wire to pin 10 of the Pi Zero which is initialized to receive inputs. When the button is pressed, the circuit is connected, and a high signal is passed to the Pi, which indicates that a photograph should be taken.

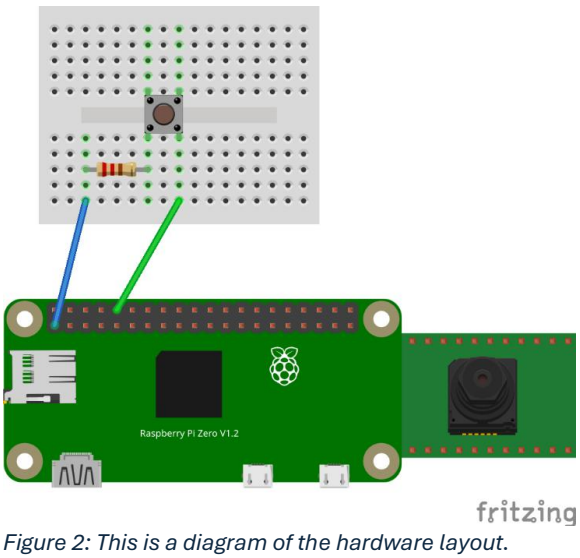

Figure 2: This is a diagram of the hardware layout.

Figure 2 displays the hardware layout. Fritzing does not have a module for a Pi Zero with a camera attachment, so I have added the camera manually. In the actual implementation, it is connected with a ribbon to the Pi Zero which has a built-in connection point for camera ribbons.

Software Solution

The functionality of this device is written in Python. The program that takes the photographs and generates the descriptions primarily uses the Time [5], PiCamZero [6], RPI.GPIO [7], mpg123 [8], and OpenAI [9] modules to provide functionality.

When the program initializes, it checks for a command line argument from the user. If there is an argument, it multiplies the number entered by 60 since the number entered is the number of minutes the user wants each interval to be, and the Time library uses seconds. If there is no argument, there is no interval at which photographs are taken, and they can only be taken when the button is pressed. If there is an interval, the button can still be pressed to take photographs. The program runs a loop and checks to see if the time since the last photograph taken has surpassed the interval. Once it does, a photograph is taken.

The button uses RPI.GPIO which is a built-in library for Raspberry Pi that allows Python to connect to the GPIO pins on the Pi. Pin 10 is specified as an input pin by the program on startup. When a high voltage reading comes from the pin indicating that the button has been pressed, a photograph is taken.

The photographs are taken using the PiCamZero library. This is another built-in library for Raspberry Pi that allows connected cameras to be controlled. Once the photograph is taken, it is saved at the “PhotosDescriptions” directory.

Once a photograph has been taken, the program uses OpenAI’s library to send it to the GPT 4-o API, which then analyzes the image and provides a text description of the image. It has been prompted to be concise. It should keep the description limited to three sentences, with two sentences describing what is in the foreground of the image and one describing the background. GPT returns a text description of the image which is saved in the “PhotosDescriptions” directory. This description is then sent to the GPT 4-o text-to-speech model which creates an audio description of the text. That audio description is also stored in the “PhotosDescriptions” directory.

The exact prompt given to the GPT API was the result of various iterations and tests. At first, I had just told the LLM to describe the image. This vague prompt worked, however, the AI would sometimes write long descriptions and focus too much on insignificant parts of the image. Long descriptions became a problem when generating text to speech and playing the audio back to the user. These overly long descriptions harmed the runtime of the program. Next, I tried telling the LLM to limit its response to two sentences. In testing, this resulted in quicker turnaround times for results but also led to many details, particularly contextual or background details, being left out of the

description. These incomplete descriptions harmed the ability for the program to be used as an automated journal, as the user would have to go back and analyze the images themselves. My next idea was to specify that the LLM should return one sentence about the foreground and one about the background. This worked fairly well, however it would still end up leaving out important details. The prompt I finally ended on told the LLM to write three sentences, with two sentences being about the foreground and one being about the background. This allowed all of the details of the foreground to be portrayed and limited the overall length of the response. I also tried two sentences of foreground and two sentences of background, but I found that in most cases there was not enough going on in the background to warrant two sentences. Ultimately, I settled on the three-sentence prompt which captured enough detail while not increasing the runtime too much.

Finally, the program uses mpg123 which is a library that allows python to play audio files to play the audio description of the image. It is played over the device's default audio output device.

The final implementation detail is the webserver. It is a simple webserver that runs using the Flask library. [10] It simply displays the audio, text, and photographs together using an HTML template. When the files are saved, they are saved under the same name, which is the exact timestamp at which the photograph was taken. This way, the webserver can easily display them together.

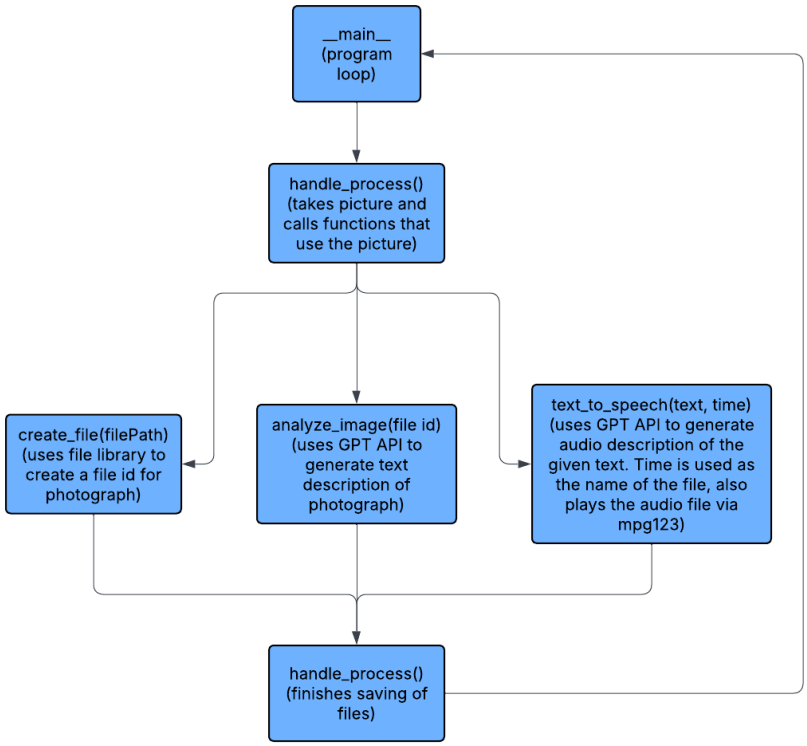

Figure 3: This diagram shows the structure of the code.

All of the code written for this project is located in the project's GitHub repository. A diagram showing the structure of the program is shown by figure 3.

Implementation Challenges

There were a number of challenges I faced while implementing this project. The first and most predictable problem was hardware related. I had originally wanted to use an Arduino development board, however, they lack the ability to have audio files sent to them in real time. This meant I would not have been able to play the audio descriptions back to the user. To fix this problem, I used the Raspberry Pi Zero. The tradeoff was that the Pi Zero is larger than the Arduino board, which in a final product would make the device slightly more difficult for the user to carry throughout the day.

A second issue arose when I wrote certain parts of my code on my laptop with the plan of later transferring the code to the Pi. Writing the code on the laptop had the benefit of providing a more comfortable development environment due to the many quality of life features and extensions Visual Studio Code offers. Transferring the code to the Pi was more difficult than I imagined it would be. The Pi Zero does not have enough processing power to open a web browser, and normally I would email or Slack myself small code segments to transfer them between devices. Instead, I ended up using FileZilla [11] and FTP to transfer the files directly from my laptop to the Pi. This worked well and was actually faster than using Slack or sending an email would have been if either were possible.

A problem that popped up during the implementation of the webserver was how I would store the three different files in such a way that I would easily be able to tell that the files went together. I had initially planned on just having a single variable as the name of all three files so that they all had the same name. I would then increment this value whenever a new photograph was taken. This created the problem of overwriting previously taken photographs if the program was stopped and started again. This ruined the idea of creating a visual diary by erasing entries. My solution to this problem was to use the current time as the name of the files. This time was recorded when the photograph was taken and used as the name of the files. This worked well and in a future iteration of the project could even allow the webserver to display more information like a time stamp.

The final problem I faced in implementing the project was that I was not sure how to display the images, text, and audio to the user for the visual journal aspect of the project. I had no prior experience in displaying files located on one device on an entirely different device. One suggestion I received from a recruiter was to use Amazon Web Server Buckets [12] to store the data on the cloud. I looked into this solution, and it seemed like a good way to implement the project. There were two problems with this implementation. One was that it would cost money. The second was that Amazon Web Servers happened to have a massive outage on the day I had dedicated to implementing this part of my project. After researching possible ways to implement this, I found Flask

and decided to use the Raspberry Pi as a web server to display the images on the local network. This was a free option and allowed the photographs to be viewed from any other devices on the network without them needing to run any software locally. It ended up being an ideal solution.

Conclusion

This project ended up being a success. I set out to create a device that could take pictures, store them to be viewed as a visual journal, and relay an audio description of the image to the user. I successfully implemented each aspect of my original plan. In the future, if I were to iterate on this project, I would like to focus on processing time of the images. Finding a way to speed up the photographs being taken and the audio descriptions being generated would make the device much more practical.

Bibliography

[1] abc. 2022. abc7. What is BeReal? What parents need to know about popular new app. (August 2022). Retrieved from https://abc7.com/post/bereal-be-real-app-what-is-social-media/12109439/.

[2] Raspberry Pi. 2025. Raspberry Pi Zero 2 W. Retrieved from https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/.

[3] adafruit. 2025. Zero Spy Camera for Raspberry Pi Zero - 120 Degree Focal Angle. Retrieved from https://www.adafruit.com/product/5389#tutorials.

[4] DFROBOT. 2025. Mini Push Button Switch (5 pcs). Retrieved from https://www.dfrobot.com/product-612.html?.

[5] Python. 2025. Python Software Foundation. time — Time access and conversions. (November 2025). Retrieved from https://docs.python.org/3/library/time.html.

[6] PiCamZero. 2024. Raspberry Pi Foundation. Picamera zero (picamzero). (November 2024). Retrieved from https://raspberrypifoundation.github.io/picamera-zero/.

[7] Croston. 2022. Python Software Foundation. RPi.GPIO 0.7.1. (February 2022). Retrieved from https://pypi.org/project/RPi.GPIO/.

[8] mpg123. 2025. mpg123 - Fast console MPEG Audio Player and decoder library. (October 2025). Retrieved from https://www.mpg123.de/.

[9] OpenAI. 2025. Libraries. Retrieved from https://platform.openai.com/docs/libraries.

[10] Pallets. 2010. Flask. Retrieved from https://flask.palletsprojects.com/en/stable/.

[11] FileZilla. 2025. Overview. (September 2025). Retrieved from https://filezilla-project.org/.

[12] aws. 2025. Amazon Web Services. Cloud Object Storage - Amazon S3. Retrieved from https://aws.amazon.com/pm/serv-s3/.